

Estructura de Seguridad en el Módulo Purchase

1 Nivel de Modelo (Model Access Rights)

Definición de Grupos de Seguridad

En `purchase/security/purchase_security.xml`:

xml

XML

```
<!-- Categoría de aplicación -->

<record id="module_category_purchase"
model="ir.module.category">

    <field name="name">Purchases</field>

    <field name="sequence">4</field>

</record>

<!-- Grupo de Usuario -->

<record id="group_purchase_user" model="res.groups">

    <field name="name">User: Own Documents Only</field>

    <field name="category_id"
ref="module_category_purchase"/>

    <field name="implied_ids" eval="[(4,
ref('base.group_user'))]">

</record>

<!-- Grupo de Manager -->
```

```

<record id="group_purchase_manager" model="res.groups">
    <field name="name">Administrator</field>

    <field name="category_id"
ref="module_category_purchase"/>

    <field name="implied_ids" eval="[(4,
ref('group_purchase_user'))]" />

    <field name="users" eval="[(4,
ref('base.user_admin'))]" />

</record>

```

Se definen en: `purchase/security/ir.model.access.csv`

Ejemplo del archivo CSV:

csv

None

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
```

```
access_purchase_order_user,purchase.order.user,model_purchase_order,purchase.group_purchase_user,1,1,1,0
```

```
access_purchase_order_manager,purchase.order.manager,model_purchase_order,purchase.group_purchase_manager,1,1,1,1
```

```
access_purchase_order_line_user,purchase.order.line.user,  
model_purchase_order_line,purchase.group_purchase_user,1,  
1,1,0
```

```
access_purchase_order_line_manager,purchase.order.line.ma  
nager,model_purchase_order_line,purchase.group_purchase_m  
anager,1,1,1,1
```

Explicación:

- Usuario de Compras (**group_purchase_user**): Puede leer, escribir y crear órdenes de compra, pero NO eliminar (0)
- Administrador de Compras (**group_purchase_manager**): Tiene todos los permisos CRUD (1,1,1,1)

2 Nivel de Registro (Record Rules)

Se definen en: **purchase/security/purchase_security.xml**

Ejemplo de regla de registro:

xml

XML

```
<record id="purchase_order_comp_rule" model="ir.rule">  
    <field name="name">Purchase Order:  
multi-company</field>  
    <field name="model_id" ref="model_purchase_order"/>  
    <field name="domain_force">  
        ['|'],
```

```

        ('company_id', '=', False),
        ('company_id', 'in', company_ids)
    ]

</field>

<field name="groups" eval="[(4,
ref('base.group_user'))]" />

</record>

<record id="purchase_order_personal_rule"
model="ir.rule">

    <field name="name">Personal Orders</field>

    <field name="model_id" ref="model_purchase_order" />

    <field name="domain_force">

        [ ('user_id', '=', user.id) ]

    </field>

    <field name="groups" eval="[(4,
ref('purchase.group_purchase_user'))]" />

</record>

<record id="purchase_order_see_all" model="ir.rule">

    <field name="name">All Orders</field>

    <field name="model_id" ref="model_purchase_order" />

    <field name="domain_force">[(1, '=', 1)]</field>

```

```
<field name="groups" eval="[(4,
ref('purchase.group_purchase_manager'))]" />

</record>
```

Explicación:

- Regla multicompañía: Los usuarios solo ven órdenes de compra de su(s) compañía(s)
- Regla personal: Los usuarios normales solo ven sus propias órdenes (`user_id = user.id`)
- Regla de administrador: Los managers ven TODAS las órdenes (`1=1` = sin filtro)

③ Nivel de Campo (Field Level Security)

Se definen en: `purchase/models/purchase.py`

Ejemplo en el modelo Python:

python

Python

```
class PurchaseOrder(models.Model):

    _name = 'purchase.order'

    _description = 'Purchase Order'

    name = fields.Char('Order Reference', required=True)
```

```
partner_id = fields.Many2one('res.partner',
string='Vendor')
```

Campo visible solo para managers

```
notes = fields.Text(

    'Terms and Conditions',

    groups="purchase.group_purchase_manager"

)
```

Campo de costo interno

```
internal_cost = fields.Float(

    'Internal Cost',

    groups="purchase.group_purchase_manager"

)
```

Ejemplo en vista XML:

xml

XML

```
<record id="purchase_order_form" model="ir.ui.view">

    <field name="name">purchase.order.form</field>

    <field name="model">purchase.order</field>

    <field name="arch" type="xml">

        <form>

            <sheet>

                <group>

                    <field name="name" />

                    <field name="partner_id" />

                    <!-- Campo visible solo para managers -->

                    <field name="notes"

groups="purchase.group_purchase_manager" />
```

```
normales --> <!-- Campo oculto para usuarios normales -->

<field name="internal_cost"

groups="purchase.group_purchase_manager"

optional="hide"/>

</group>

</sheet>

</form>

</field>

</record>
```



Ejercicios de Seguridad en Odoo

Módulo: solt_library



EJERCICIO 1: Crear Grupo Básico "Lector"

Objetivo: Crear un nuevo grupo que solo pueda leer libros y autores

Descripción del Problema

La biblioteca quiere permitir que usuarios públicos puedan **consultar** el catálogo de libros y autores, pero **sin** capacidad de crear, modificar o eliminar registros.

Tareas

1. Crear un nuevo grupo llamado `library_group_reader` (Lector)
2. Este grupo debe aparecer en el menú de Configuración → Usuarios → Grupos
3. El grupo debe tener categoría "Biblioteca"

Solución

Archivo: `security/security.xml`

XML

```
<?xml version="1.0" encoding="utf-8"?>
.....

    <!-- NUEVO: Grupo Lector -->
    <record id="library_group_reader" model="res.groups">
        <field name="name">Reader</field>
        <field name="category_id" ref="module_category_library
    "/>
    ....
```

Actualizar: `security/ir.model.access.csv`

None

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
access_library_book_reader,library.book.reader,model_library_book,library_group_reader,1,0,0,0
access_library_author_reader,library.author.reader,model_library_author,library_group_reader,1,0,0,0
access_library_category_reader,library.category.reader,model_library_category,library_group_reader,1,0,0,0
```

Verificación

1. Ir a Configuración → Usuarios → Grupos
2. Buscar "Biblioteca" en categorías
3. Asignar grupo "Lector" a un usuario de prueba
4. Iniciar sesión con ese usuario
5. Verificar que puede ver libros pero no crear/modificar/eliminar



EJERCICIO 2: Permisos CRUD Diferenciados por Rol

Objetivo: Implementar permisos CRUD completos para todos los modelos según los roles

Descripción del Problema

Actualmente los permisos están incompletos. Se necesita:

- **Lector:** Solo lectura en libros, autores, categorías
- **Usuario:** Lectura en todos + crear préstamos propios
- **Bibliotecario:** CRUD completo en libros, autores, préstamos. Solo lectura en miembros
- **Gerente:** CRUD completo en todo

Solución

Archivo: `security/ir.model.access.csv` (completo)

None

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
"LECTOR - Solo Lectura",,,,,,
access_library_book_reader,library.book.reader,model_library_book,library_group_reader,1,0,0,0
access_library_author_reader,library.author.reader,model_library_author,library_group_reader,1,0,0,0
access_library_category_reader,library.category.reader,model_library_category,library_group_reader,1,0,0,0
"USUARIO - Lectura + Crear Préstamos",,,,,,
access_library_book_user,library.book.user,model_library_book,library_group_user,1,0,0,0
```

```

access_library_author_user,library.author.user,model_library_a
uthor,library_group_user,1,0,0,0
access_library_category_user,library.category.user,model_libra
ry_category,library_group_user,1,0,0,0
access_library_member_user,library.member.user,model_library_m
ember,library_group_user,1,0,0,0
access_library_loan_user,library.loan.user,model_library_loan,
library_group_user,1,0,1,0
"BIBLIOTECARIO - CRUD Libros/Autores/Préstamos",,,,,,
access_library_book_librarian,library.book.librarian,model_lib
rary_book,library_group_librarian,1,1,1,1
access_library_author_librarian,library.author.librarian,model
_library_author,library_group_librarian,1,1,1,1
access_library_category_librarian,library.category.librarian,m
odel_library_category,library_group_librarian,1,1,1,1
access_library_loan_librarian,library.loan.librarian,model_lib
rary_loan,library_group_librarian,1,1,1,1
access_library_member_librarian,library.member.librarian,model
_library_member,library_group_librarian,1,0,0,0
"GERENTE - CRUD Completo",,,,,,
access_library_book_manager,library.book.manager,model_library
_book,library_group_manager,1,1,1,1
access_library_author_manager,library.author.manager,model_lib
rary_author,library_group_manager,1,1,1,1
access_library_category_manager,library.category.manager,model
_library_category,library_group_manager,1,1,1,1
access_library_member_manager,library.member.manager,model_lib
rary_member,library_group_manager,1,1,1,1
access_library_loan_manager,library.loan.manager,model_library
_loan,library_group_manager,1,1,1,1

```

Matriz de Permisos

Modelo	Lector	Usuario	Bibliotecario	Gerente
library.book	R	R	CRUD	CRUD
library.author	R	R	CRUD	CRUD
library.category	R	R	CRUD	CRUD

library.member	-	R	R	CRUD
library.loan	-	R+C	CRUD	CRUD

Verificación

Crear 4 usuarios de prueba (uno por rol) y verificar que:

- Lector: Solo puede ver libros/autores
- Usuario: Puede crear préstamos pero no modificarlos
- Bibliotecario: Puede gestionar todo excepto miembros
- Gerente: Acceso total



EJERCICIO 3: Record Rules - Ver Solo Mis Préstamos

Objetivo: Los usuarios normales solo deben ver sus propios préstamos

Descripción del Problema

Actualmente, cualquier usuario con acceso a préstamos puede ver **todos** los préstamos del sistema. Esto viola la privacidad. Se necesita que:

- **Usuarios normales:** Solo vean préstamos donde ellos son el miembro
- **Bibliotecarios y Gerentes:** Vean todos los préstamos

Solución

Archivo: `security/security.xml` (agregar al final)

XML

```
<!-- Record Rule: Ver solo mis préstamos -->
<record id="library_loan_personal_rule" model="ir.rule">
  <field name="name">Préstamos Personales</field>
  <field name="model_id" ref="model_library_loan"/>
  <field
name="domain_force">[( 'member_id.partner_id.user_ids', 'in',
user.id)]</field>
  <field name="groups" eval="[(4,
ref('library_group_user'))]" />
</record>
```

```

        <field name="perm_read" eval="True"/>
        <field name="perm_write" eval="False"/>
        <field name="perm_create" eval="True"/>
        <field name="perm_unlink" eval="False"/>
    </record>

    <!-- Record Rule: Bibliotecarios y Gerentes ven todo -->
    <record id="library_loan_all_rule" model="ir.rule">
        <field name="name">Todos los Préstamos</field>
        <field name="model_id" ref="model_library_loan"/>
        <field name="domain_force">[(1, '=', 1)]</field>
        <field name="groups" eval="[(4,
ref('library_group_librarian'))]">
        <field name="perm_read" eval="True"/>
        <field name="perm_write" eval="True"/>
        <field name="perm_create" eval="True"/>
        <field name="perm_unlink" eval="True"/>
    </record>

```

Explicación del Dominio

Dominio para usuarios:

Python

```
[('member_id.partner_id.user_ids', 'in', user.id)]
```

- `member_id`: El miembro que realizó el préstamo
- `.partner_id`: El contacto asociado al miembro
- `.user_ids`: Los usuarios vinculados a ese contacto
- `in user.id`: El usuario actual está en esa lista

Dominio para bibliotecarios:

Python

```
[(1, '=', 1)] # Siempre verdadero = ver todo
```

Caso de Prueba

Configuración:

1. Usuario Juan (usuario normal, member_id=1)
2. Usuario María (usuario normal, member_id=2)
3. Usuario Pedro (bibliotecario)

Datos de prueba:

- Préstamo #1: member_id=1 (Juan) - Libro "Don Quijote"
- Préstamo #2: member_id=2 (María) - Libro "Cien Años de Soledad"
- Préstamo #3: member_id=1 (Juan) - Libro "El Principito"

Resultados esperados:

- Juan ve: Préstamo #1 y #3
- María ve: Préstamo #2
- Pedro ve: Todos los préstamos

Verificación

```
Python
# Desde consola Python de Odoo
# Como Juan (user_id=X)
loans = env['library.loan'].sudo(user_juan).search([])
print(loans.mapped('reference')) # Debe mostrar solo
préstamos de Juan
```



EJERCICIO 4: Record Rules Complejas - Préstamos por Compañía

Objetivo: Multi-compañía - cada bibliotecario solo gestiona préstamos de su sucursal

Descripción del Problema

La biblioteca tiene múltiples sucursales (compañías en Odoo). Los bibliotecarios deben:

- Ver/gestionar solo préstamos de su compañía
- Los gerentes ven todo

Preparación del Modelo

Archivo: `models/library_loan.py`

```
Python
# Agregar campo de compañía
company_id = fields.Many2one(
    'res.company',
    string='Compañía',
    default=lambda self: self.env.company,
    required=True
)
```

Actualizar vista: `views/library_loan_views.xml`

```
XML
<!-- Agregar en el formulario -->
<field name="company_id" groups="base.group_multi_company"/>
```

Solución

Archivo: `security/security.xml`

```
XML
<!-- Record Rule: Bibliotecarios - Solo su compañía -->
<record id="library_loan_company_rule" model="ir.rule">
    <field name="name">Préstamos por Compañía
(Bibliotecario)</field>
    <field name="model_id" ref="model_library_loan"/>
    <field name="domain_force">['|',
        ('company_id', '=', False),
        ('company_id', 'in', company_ids)
    ]</field>
    <field name="groups" eval="[(4,
ref('library_group_librarian'))]">
    </record>

<!-- Record Rule: Usuarios - Su compañía Y sus préstamos
-->
```

```

    <record id="library_loan_user_company_rule"
model="ir.rule">
    <field name="name">Mis Préstamos en Mi
Compañía</field>
    <field name="model_id" ref="model_library_loan"/>
    <field name="domain_force">['&',
        '|',
        ('company_id', '=', False),
        ('company_id', 'in', company_ids),
        ('member_id.partner_id.user_ids', 'in', user.id)
    ]</field>
    <field name="groups" eval="[(4,
ref('library_group_user'))]" />
</record>

<!-- Record Rule: Gerentes - Todas las compañías -->
<record id="library_loan_manager_all_rule"
model="ir.rule">
    <field name="name">Todos los Préstamos
(Gerente)</field>
    <field name="model_id" ref="model_library_loan"/>
    <field name="domain_force">[(1, '=', 1)]</field>
    <field name="groups" eval="[(4,
ref('library_group_manager'))]" />
</record>

```

Explicación de Dominios

Para Bibliotecarios:

```

Python
['|',
    ('company_id', '=', False),          # Sin compañía asignada
    ('company_id', 'in', company_ids) # 0 en compañías del
usuario
]

```

Para Usuarios (combinación AND):

Python

```
['&', # AND principal
 '|', # Primera condición: compañía válida
    ('company_id', '=', False),
    ('company_id', 'in', company_ids),
    ('member_id.partner_id.user_ids', 'in', user.id) #
Segunda: es su préstamo
]
```

Caso de Prueba

Configuración Multi-Compañía:

- Compañía A: "Biblioteca Central"
- Compañía B: "Biblioteca Norte"

Usuarios:

- Ana (Bibliotecaria, Compañía A)
- Luis (Bibliotecario, Compañía B)
- Roberto (Gerente, ambas compañías)

Datos:

- Préstamo #1: Compañía A, member=Juan
- Préstamo #2: Compañía B, member=María
- Préstamo #3: Compañía A, member=Pedro

Resultados esperados:

- Ana ve: Préstamo #1 y #3
- Luis ve: Préstamo #2
- Roberto ve: Todos



EJERCICIO 5: Seguridad a Nivel de Campo - Información Sensible

Nivel: Avanzado - Campo groups y compute_sudo

Objetivo: Ocultar información sensible de miembros según el rol

Descripción del Problema

Los miembros tienen información financiera sensible (multas pendientes, historial de pagos). Esta información debe ser:

- **Invisible** para usuarios normales
- **Visible pero no editable** para bibliotecarios
- **Completamente editable** solo para gerentes

Preparación del Modelo

Archivo: `models/library_member.py`

Python

```
from odoo import models, fields, api

class LibraryMember(models.Model):
    _inherit = 'library.member'

    # Campos sensibles - Solo gerentes pueden ver/editar
    credit_limit = fields.Monetary(
        'Límite de Crédito',
        currency_field='currency_id',
        groups='solt_library.library_group_manager', # Solo
gerentes
        help="Monto máximo de multas antes de suspender"
    )

    total_fines_paid = fields.Monetary(
        'Total Multas Pagadas',
        currency_field='currency_id',
        compute='_compute_fines_history',
        groups='solt_library.library_group_librarian', #
Bibliotecarios+
        help="Suma histórica de todas las multas pagadas"
    )

    # Campo visible para bibliotecarios pero calculado con
sudo
    pending_fees_details = fields.Text(
        'Detalle de Multas',
        compute='_compute_pending_fees_details',
        groups='solt_library.library_group_librarian'
    )
```

```

# Campo que todos pueden ver pero solo gerente edita
notes = fields.Text(
    'Notas Administrativas',
    readonly=True,
    groups='solt_library.library_group_user'
)

@api.depends('loan_ids.late_fee', 'loan_ids.fee_paid')
def _compute_fines_history(self):
    """Calcula histórico de multas - acceso restringido"""
    for member in self:
        paid_loans = member.loan_ids.filtered('fee_paid')
        member.total_fines_paid =
sum(paid_loans.mapped('late_fee'))

@api.depends('loan_ids.state', 'loan_ids.late_fee')
def _compute_pending_fees_details(self):
    """Detalle de multas pendientes"""
    for member in self.sudo(): # sudo para acceder a
préstamos
        overdue_loans = member.loan_ids.filtered(
            lambda l: l.state == 'confirmed' and
l.days_overdue > 0
        )
        if overdue_loans:
            details = []
            for loan in overdue_loans:
                details.append(
                    f"• {loan.book_id.name}:
${loan.late_fee} "
                    f"({loan.days_overdue} días)"
                )
            member.pending_fees_details =
'\n'.join(details)
        else:
            member.pending_fees_details = "Sin multas
pendientes"

```

Configuración de Vista

Archivo: `views/library_member_views.xml`

XML

```
<record id="view_library_member_form" model="ir.ui.view">
  <field name="name">library.member.form</field>
  <field name="model">library.member</field>
  <field name="arch" type="xml">
    <form>
      <sheet>
        <group>
          <group>
            <field name="partner_id"/>
            <field name="code"/>
            <field name="membership_type"/>
          </group>
          <group>
            <!-- Visible para todos, editable solo
gerente -->
            <field name="notes"
readonly="context.get('default_readonly', True)"
            attrs="{ 'readonly': [('id',
'!=', False)]}" />
          </group>
        </group>

        <!-- Notebook con información sensible -->
        <notebook>
          <page string="Préstamos Activos">
            <field name="loan_ids"/>
          </page>

          <!-- Solo visible para Bibliotecarios+ -->
          <page string="Información Financiera"

groups="solt_library.library_group_librarian">
          <group>
            <field name="pending_fees"/>
          </group>
        </notebook>
      </sheet>
    </form>
  </field>
</record>
```

```

                <field name="pending_fees_details"
                    widget="text"
                    readonly="1"/>
                <field name="total_fines_paid"
                    readonly="1"/>
            </group>
        </page>

        <!-- Solo visible para Gerentes -->
        <page string="Administración"

groups="solt_library.library_group_manager">
    <group>
        <field name="credit_limit"/>
        <field name="state"/>
    </group>
</page>
</notebook>
</sheet>
</form>
</field>
</record>

```

Seguridad Adicional en Métodos

Archivo: `models/library_member.py`

Python

```

class LibraryMember(models.Model):
    _inherit = 'library.member'

    def action_suspend_membership(self):
        """Solo gerentes pueden suspender membresías"""
        self.ensure_one()
        if not
self.env.user.has_group('solt_library.library_group_manager'):
            raise AccessError("Solo los gerentes pueden
suspender membresías")

```

```

        self.state = 'suspended'
        # Notificar al usuario...

    def write(self, vals):
        """Validaciones adicionales de seguridad"""
        # Solo gerentes pueden modificar credit_limit
        if 'credit_limit' in vals:
            if not
self.env.user.has_group('solt_library.library_group_manager'):
                raise AccessError(
                    "Solo los gerentes pueden modificar el
límite de crédito"
                )

            # Solo bibliotecarios+ pueden cambiar estado
            if 'state' in vals:
                if not
self.env.user.has_group('solt_library.library_group_librarian'
):
                    raise AccessError(
                        "No tiene permisos para cambiar el estado
de membresía"
                    )

        return super().write(vals)

```

Caso de Prueba

Usuarios:

- Usuario Normal: Carlos
- Bibliotecario: Laura
- Gerente: Santiago

Verificación:

Campo	Carlos	Laura	Santiago
pending_fees	 No ve	 Ve (readonly)	 Ve + Edita

pending_fees_details	✗ No ve	✓ Ve	✓ Ve
total_fines_paid	✗ No ve	✓ Ve	✓ Ve
credit_limit	✗ No ve	✗ No ve	✓ Ve + Edita
notes	✓ Ve (readonly)	✓ Ve (readonly)	✓ Ve + Edita

Documentación Oficial:

- [Security in Odoo](#)
- [Access Rights](#)
- [Record Rules](#)